

---

**FROM HUMAN TEXT TO SQL: AUTOMATING FINANCIAL  
DATA QUALITY QUERIES FOR ANOMALY REMEDIATION**

---

JULES AUBIGNAT - ARTHUR DANJOU -  
AXELLE MERIC - MORITZ VON SIEMENS

COURSE SUPERVISORS:

KATIA MEZIANI

ASSISTANT PROFESSOR (MAÎTRE DE CONFÉRENCES) - CEREMADE

DIDIER JEANNEL

SAFRAN

THÉO LOPES-QUINTAS

BPCE PAYMENT SERVICES

## **Abstract**

To support Natixis in managing its golden source database, we developed an AI assistant that facilitates the use of KURE, the internal data-quality platform. The assistant helps users identify key anomaly indicators in order to prioritize remediation efforts and improve data reliability. On the backend, we implemented a secure TypeScript/Vue API layer connected to MySQL data sources, ensuring robust schema grounding, reliable SQL execution, and context-aware responses. Through model selection and guided interactions, we aimed to balance domain expertise and automation by keeping users in control while providing a simple and effective conversational experience.

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>3</b>  |
| <b>2</b> | <b>Project Objectives and Scope</b>                     | <b>3</b>  |
| <b>3</b> | <b>Data Pipeline and Database Construction</b>          | <b>3</b>  |
| <b>4</b> | <b>System Architecture</b>                              | <b>3</b>  |
| 4.1      | Model Orchestration and Integration . . . . .           | 4         |
| 4.2      | Tool-Based Execution Protocol . . . . .                 | 4         |
| 4.3      | Translation Layer and Future Foundations . . . . .      | 5         |
| <b>5</b> | <b>System Prompt Design and Architecture</b>            | <b>6</b>  |
| 5.1      | Persona Definition and Contextual Framing . . . . .     | 6         |
| 5.2      | Schema Grounding and Hallucination Prevention . . . . . | 6         |
| 5.3      | Canonical Business Logic Injection . . . . .            | 6         |
| 5.4      | Execution Protocol and Tool Orchestration . . . . .     | 6         |
| 5.5      | Adaptive Multilingual Management . . . . .              | 7         |
| <b>6</b> | <b>User Interface and User Experience</b>               | <b>7</b>  |
| 6.1      | Human-in-the-Loop and Clarification Phase . . . . .     | 9         |
| 6.2      | Progressive Disclosure of Execution Steps . . . . .     | 10        |
| 6.3      | Multimodal Output and Results Interpretation . . . . .  | 11        |
| <b>7</b> | <b>System Evaluation and Limitations</b>                | <b>14</b> |
| 7.1      | Evaluation of Text-to-SQL Accuracy . . . . .            | 14        |
| 7.2      | Architectural Limitations . . . . .                     | 14        |
| <b>8</b> | <b>Conclusion</b>                                       | <b>14</b> |

# 1 Introduction

With the introduction of KURE, Natixis addressed a major challenge in financial data governance, namely anomalies in the golden source and information discrepancies across teams and systems. Because standards and regulatory requirements evolve continuously, data quality management must remain active over time to preserve accuracy, consistency, completeness, reliability, and relevance throughout the organization. In this context, our project aims to make anomaly data directly accessible through natural language. Instead of writing SQL manually, users can ask business-oriented questions and obtain immediate answers generated from the underlying relational database. This approach reduces the technical barrier to data exploration and accelerates the identification of recurrent or critical anomaly patterns. From a technical perspective, the first phase focused on establishing a reliable data foundation. We developed `data_exploration.ipynb` to inspect, clean, and transform the source files, then prepared the SQL import pipeline. We defined the schema in `init.sql`, deployed a MySQL instance with Docker, created the required tables, and loaded data from files stored in `./data`. This process ensured alignment between raw files, relational structure, and downstream chatbot behavior.

## 2 Project Objectives and Scope

The objective of the assistant is to provide decision-oriented access to anomaly information without requiring end users to master SQL syntax or database internals. The application was designed to answer operational questions on anomaly volume, severity, ownership, and evolution while maintaining consistency with Natixis business rules. The functional scope includes natural-language question interpretation, SQL query generation, query execution against a controlled schema, and structured response rendering. The project deliberately prioritizes explainability and usability so that users can understand what the system returns and act on it in KURE processes.

## 3 Data Pipeline and Database Construction

The data pipeline begins with exploratory and preparatory work in `data_exploration.ipynb`, where tabular files are inspected for data types, naming consistency, missing values, and semantic compatibility with the target schema. The transformed data is then mapped to relational entities defined in `init.sql`. A containerized MySQL service is deployed with Docker to guarantee reproducibility across development environments. Once the service is running, table creation scripts are executed, and ingestion scripts load the records from `./data` into the database. This architecture isolates storage from the application layer and makes initialization deterministic for demos, testing sessions, and iterative development.

## 4 System Architecture

The repository is primarily composed of Vue and TypeScript components, which reflects a modern full-stack orientation centered on maintainability and interface responsiveness. The architecture follows a clear separation of concerns. The user interface captures intent and provides a conversational experience inspired by familiar LLM interfaces. Simultaneously, the backend acts as a dedicated orchestration layer between the chat interface, the selected language model, and the

MySQL anomaly database. By isolating the NLP interpretation, SQL generation, and database connections, this modular design simplifies debugging and allows independent improvements to each subsystem.

## 4.1 Model Orchestration and Integration

The core role of the backend is to transform user requests into controlled analytical actions while preserving a natural conversational experience. Concretely, upon receiving a user message, the backend enriches the model context with domain and schema constraints, invokes the model through the AI SDK, and supervises the execution of tools used to produce the final answer.

A central implementation choice was the use of Ollama through the AI SDK in order to support local model execution while keeping a unified programming interface for generation, tool calling, and streaming responses. This approach gave us the flexibility to test different models with limited integration overhead, while maintaining the same backend pipeline and prompt contract. It also made it possible to balance cost, latency, and reasoning quality depending on the use case and the selected model.

## 4.2 Tool-Based Execution Protocol

The backend logic relies on a strictly defined tool-based architecture to reduce ambiguity in model behavior and improve the reproducibility of outputs. The `executeSqlTool` is responsible for executing SQL queries against MySQL and returning structured results to the model pipeline. The `chartTool` is used when the user asks for visual exploration, allowing the assistant to return chart-ready outputs instead of only textual summaries. Finally, the `kpiTool` is dedicated to key indicators and compact performance-style metrics, which is particularly useful for anomaly monitoring and operational reporting.

```

❖ executing SQL: SELECT DATE_FORMAT(ga.asof_dat, '%Y-%m') AS month,
COUNT(*) AS total_anomalies,
SUM(CASE
    WHEN UPPER(ga.priority_typ) IN ('CRITICAL','CRITIQUE','HIGH','HAUTE')
    OR ga.hotfix_flg = 1
    OR UPPER(fc.criticity_typ) IN ('CRITICAL','CRITIQUE','HIGH','HAUTE')
    THEN 1 ELSE 0 END) AS critical_anomalies,
COUNT(*) - SUM(CASE
    WHEN UPPER(ga.priority_typ) IN ('CRITICAL','CRITIQUE','HIGH','HAUTE')
    OR ga.hotfix_flg = 1
    OR UPPER(fc.criticity_typ) IN ('CRITICAL','CRITIQUE','HIGH','HAUTE')
    THEN 1 ELSE 0 END) AS non_critical_anomalies
FROM generic_anomalies ga
LEFT JOIN typologies t ON ga.typology_id = t.typology_id
LEFT JOIN functional_controls fc ON t.functional_control_id = fc.functional_control_id
WHERE ga.asof_dat BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY month
ORDER BY month
LIMIT 3000;
❖ executing SQL: SELECT t.typology_id,
COALESCE(t.typology_en_lbl, t.typology_fr_lbl, 'UNKNOWN') AS typology_label,
COUNT(*) AS anomaly_count
FROM generic_anomalies ga
LEFT JOIN typologies t ON ga.typology_id = t.typology_id
WHERE ga.asof_dat BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY t.typology_id, typology_label
ORDER BY anomaly_count DESC
LIMIT 10;
❖ executing SQL: SELECT SUM(CASE
    WHEN UPPER(ga.priority_typ) IN ('CRITICAL','CRITIQUE','HIGH','HAUTE')
    OR ga.hotfix_flg = 1
    OR UPPER(fc.criticity_typ) IN ('CRITICAL','CRITIQUE','HIGH','HAUTE')
    THEN 1 ELSE 0 END) AS critical_anomalies,
COUNT(*) - SUM(CASE
    WHEN UPPER(ga.priority_typ) IN ('CRITICAL','CRITIQUE','HIGH','HAUTE')
    OR ga.hotfix_flg = 1
    OR UPPER(fc.criticity_typ) IN ('CRITICAL','CRITIQUE','HIGH','HAUTE')
    THEN 1 ELSE 0 END) AS non_critical_anomalies,
COUNT(*) AS total_anomalies
FROM generic_anomalies ga
LEFT JOIN typologies t ON ga.typology_id = t.typology_id
LEFT JOIN functional_controls fc ON t.functional_control_id = fc.functional_control_id
WHERE ga.asof_dat BETWEEN '2025-01-01' AND '2025-12-31';
6 KPI(s) affichés
Chart tool successfully executed

```

Figure 1: Backend terminal logs demonstrating the automated execution of SQL queries and tool invocation

The system prompt and the tool layer were designed jointly. The prompt constrains the model to the known schema and business definitions, while the tools provide safe and explicit execution channels. In practice, this means that the model does not directly act on the database without supervision; it must use the dedicated SQL execution tool within the backend protocol. This architecture improves reliability and makes debugging easier, because each step of the pipeline can be inspected independently, from intent interpretation to SQL generation and result rendering.

### 4.3 Translation Layer and Future Foundations

Special care was given to response formatting and user transparency. Even when SQL is generated and executed internally, the final answer is presented in business language with interpretable conclusions. The backend therefore acts not only as an execution engine but also as a translation layer between technical operations and decision-oriented outputs.

Overall, this architecture allowed us to combine flexibility in model selection, control in query execution, and clarity in final responses. It provides a strong foundation for future improvements

such as stricter SQL validation, finer access control policies, and more advanced observability of model-tool interactions in production contexts.

## 5 System Prompt Design and Architecture

### 5.1 Persona Definition and Contextual Framing

The system prompt initialises the Large Language Model (LLM) by assigning it a specific identity: an "Expert Data Analyst & MLOps for Natixis". This persona conditions the model to adopt an analytical approach, primarily aiming to transform natural language business questions into executable SQL queries and actionable decision-making insights.

### 5.2 Schema Grounding and Hallucination Prevention

To mitigate the common issue of AI hallucinations, the prompt dynamically integrates the MySQL database schema.

- **Strict Single Source of Truth:** The model is explicitly instructed to "STRICTLY" operate only on the provided schema and is forbidden from inventing missing columns (such as `status_typ` or `declaration_time`).
- **Data Routing and Reconciliation:** The prompt acts as an intelligent query router. It designates `generic_anomalies` as the default entry point for macroscopic aggregations, while instructing the model to cross-reference with the `anomalies` table using the mandatory `anomaly_kuid` key when assessing data loading quality or population discrepancies.
- **Complex Field Management:** The agent is specifically instructed to use MySQL JSON functions to robustly query structured text fields like `object_identification_fields` and `error_fields`.

### 5.3 Canonical Business Logic Injection

A core strength of this system prompt is the pre-injection of Natixis's specific anomaly data business rules, enabling the AI to semantically understand user intents:

- **Criticality:** An anomaly is classified as critical if its `priority_type` is high, if the `hotfix_flg` is set to 1, or if the associated functional control's criticality is high.
- **Resolution Status:** Whether an anomaly is open or closed is determined by checking the `resolved_value_txt` key within the JSON lists; in its absence, a time-based proxy using `detection_time` is applied.
- **Ownership and Assignment:** The agent is programmed to target the `responsible_login_id` field in the functional controls table to determine anomaly assignment.

### 5.4 Execution Protocol and Tool Orchestration

The application leverages an automated tool-calling system featuring `executeSqlTool` for query execution, `chartTool` for visualisations, and `kpiTool` for key performance indicators. The system

prompt rigorously frames the use of these tools through a precise 7-step execution pipeline (Analysis, Clarification, Invisible SQL, Conditional KPIs, Conditional Visualization, Proactive Proposal, and Synthesis):

- **Clarification Phase:** The model must systematically pause and request clarification from the user if a query is ambiguous before executing any SQL.
- **Deterministic UI Component Orchestration:** To maintain a clean interface, strict anti-duplication rules are enforced. If the `kpiTool` or `chartTool` is triggered, the model is strictly forbidden from generating redundant Markdown tables or listing the KPIs in its text response. It must yield the rendering entirely to the Vue.js components.
- **Frontend Data Binding:** The prompt enforces Nuxt-charts data binding rules, explicitly instructing the LLM to generate SQL aliases in `snake_case` that perfectly match the expected component keys (e.g., `xKey` and `series.key`).
- **User Interface (UI) Transparency:** The prompt enforces that raw SQL code must never be displayed to the end-user and forbids the AI from mentioning the background tools, thereby maintaining a natural and transparent conversational experience.

## 5.5 Adaptive Multilingual Management

Although the system instructions are written in French, the prompt includes an "Absolute Priority" rule regarding language. The agent is forced to align its response language (including titles, summaries, and charts) exclusively with the user's last message, whether in English or French, adapting seamlessly to the user's needs.

# 6 User Interface and User Experience

While the backend orchestration and the system prompt provide a robust and secure analytical engine, this technical complexity must be entirely abstracted for the end-user. To ensure that data stewards and business analysts at Natixis can leverage these capabilities without friction, we designed an interface that prioritizes accessibility, transparency, and cognitive ease.

To grant users maximum control and flexibility, the application features a model selection dropdown. Through distinct icons, the interface explicitly indicates each model's operational paradigm (local execution versus cloud-based API), reasoning capabilities, and associated cost structures. This transparency allows users to make informed trade-offs between processing speed, operational cost, and the required depth of reasoning for their specific query.

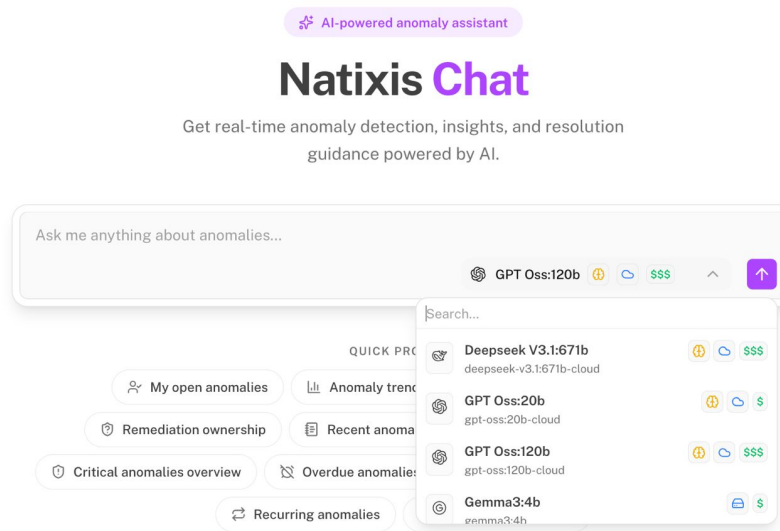


Figure 2: Interface for LLM model selection and configuration

The primary conversational interface was deliberately designed to mirror industry-standard LLM platforms like ChatGPT. Capitalizing on this established layout significantly reduces the learning curve and accelerates user onboarding. To mitigate the "blank canvas" syndrome, we implemented dynamic "Quick Prompts" below the search bar, derived from common Natixis business questions.

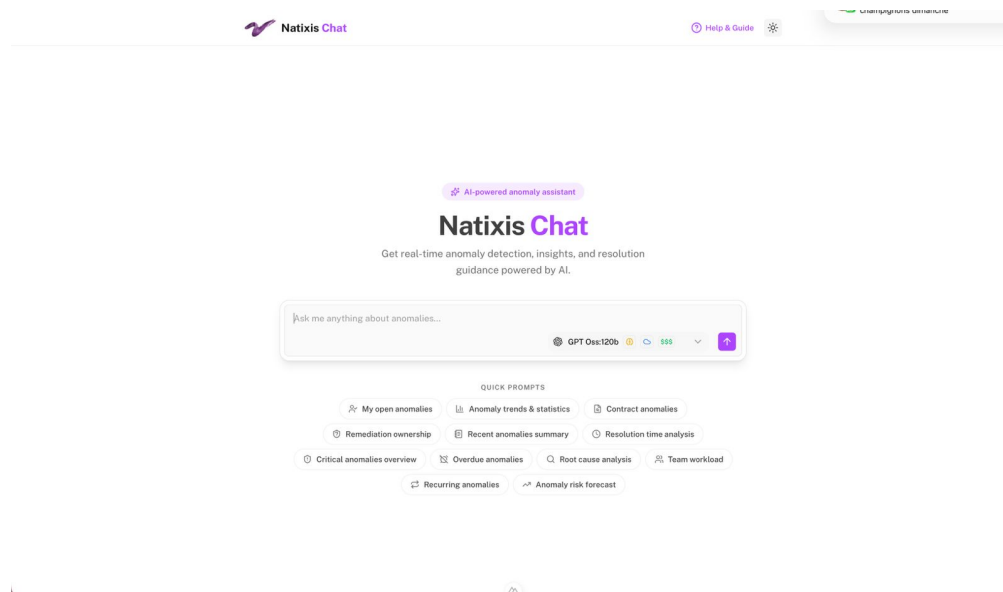


Figure 3: Main conversational interface with business-oriented prompt suggestions

Recognizing the intensive screen time typical of data analysts and financial operators, we integrated a native dark mode, improving visual ergonomics for prolonged use.

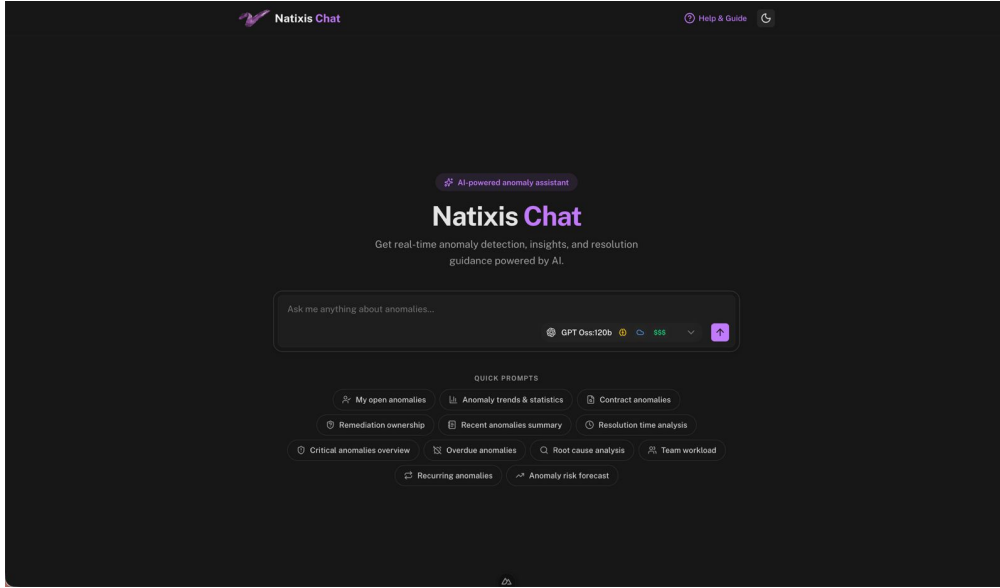


Figure 4: Dark mode interface for enhanced visual ergonomics

Trust is a critical factor in AI adoption within enterprise environments. A dedicated help window demystifies the assistant’s internal pipeline and provides practical tips on phrasing and keyword usage, empowering users to refine their prompts. Furthermore, the "Available Data" window provides a clear, high-level overview of the underlying database schema, helping users gauge whether their intended analysis falls within the current functional scope.

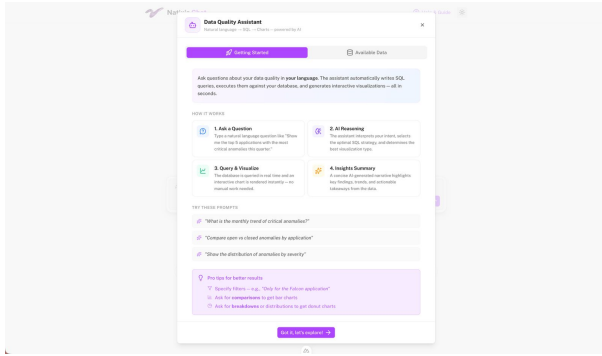


Figure 5: Help window detailing the execution pipeline

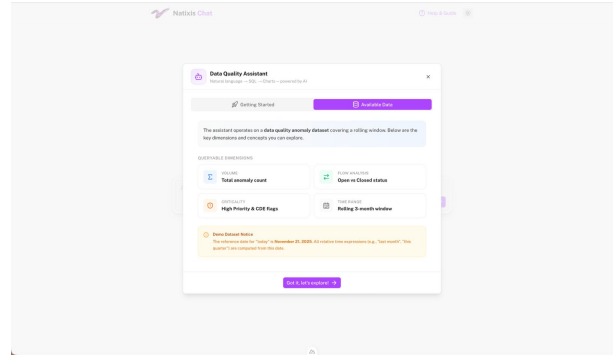


Figure 6: Available Data window summarizing queryable metrics

## 6.1 Human-in-the-Loop and Clarification Phase

A core principle of the interface is to prevent erroneous data retrieval caused by vague user requests. When an inquiry lacks necessary constraints (e.g., aggregation level, specific timeframe), the system is programmed to pause its execution pipeline and trigger a clarification request. This Human-in-the-Loop mechanism ensures that the generated SQL strictly aligns with the user’s actual analytical intent.

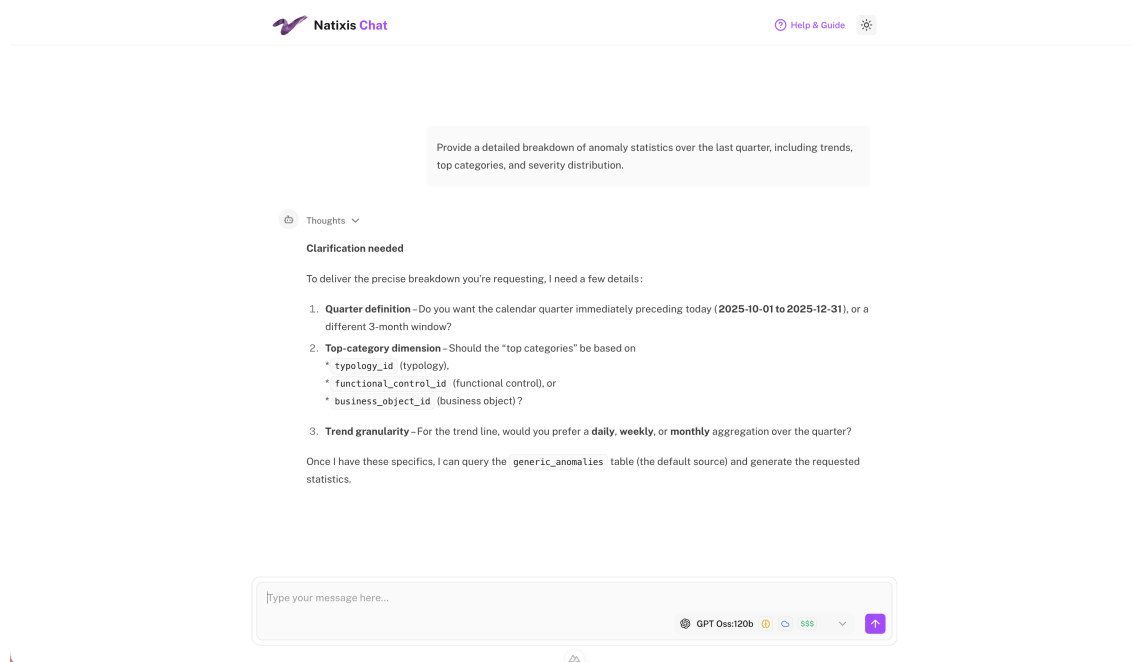


Figure 7: The model requesting clarification to resolve ambiguity before generating SQL

## 6.2 Progressive Disclosure of Execution Steps

To foster trust without overwhelming the user with raw technical data, the interface employs a progressive disclosure strategy. The reasoning process (Chain-of-Thought) and the internal tool invocations are hidden by default under expandable menus.

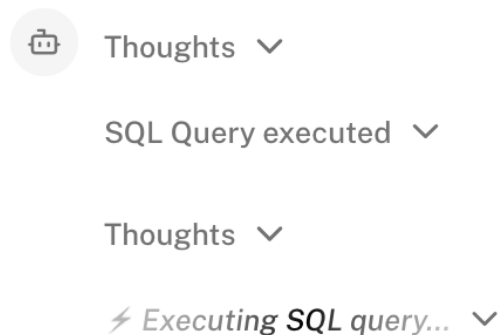


Figure 8: Collapsed view of the AI's internal reasoning and tool execution

If the user needs to audit the assistant's behavior, they can expand these sections to reveal the exact SQL query generated and executed against the database, providing complete transparency over the data extraction process.

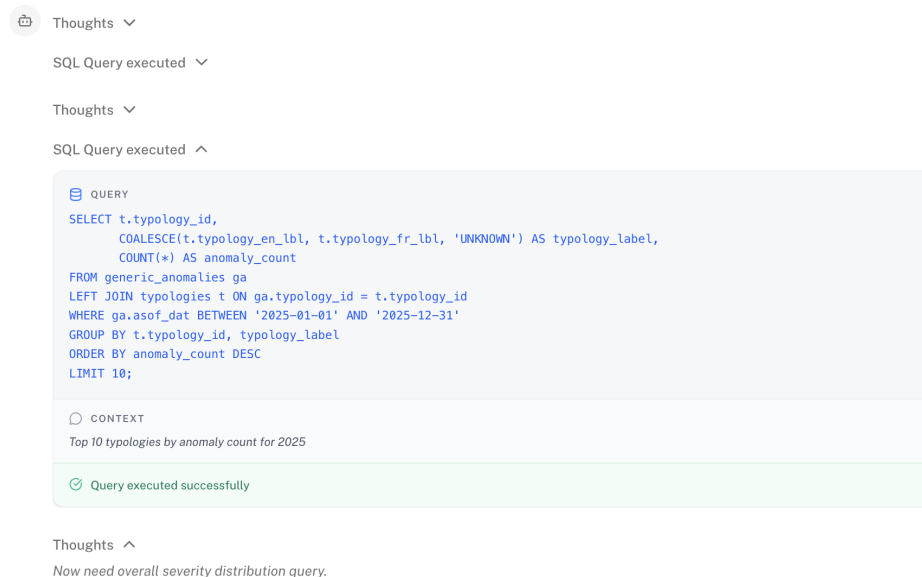


Figure 9: Expanded view revealing the generated SQL query and context

## 6.3 Multimodal Output and Results Interpretation

Instead of returning a single block of raw text, the frontend dynamically maps the backend tool responses into structured UI components. The assistant can proactively suggest optimal visualization formats based on the data retrieved.

*Would you like a visualisation (e.g., line chart of the monthly trend or a donut chart of the severity split) or a KPI-card summary?*

Figure 10: Proactive suggestion of visualization formats

When rendering the results, the interface provides a comprehensive analytical dashboard. Key metrics are highlighted using dedicated KPI cards for immediate readability.

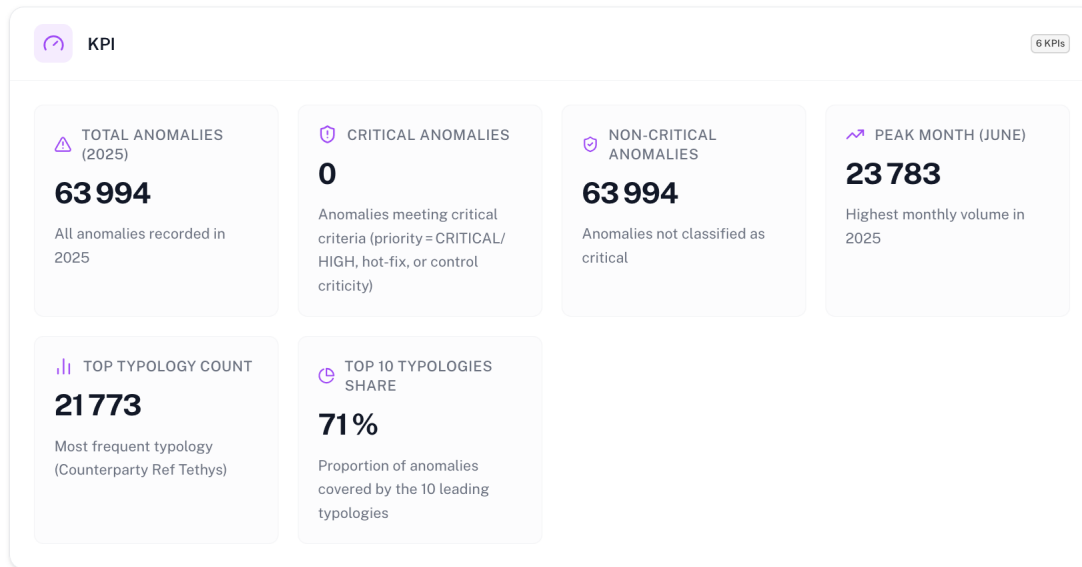


Figure 11: Structured KPI cards highlighting high-level metrics

For temporal analysis or distribution studies, the system generates interactive visualizations, such as bar plots, accompanied by tabular data formats for deeper inspection.

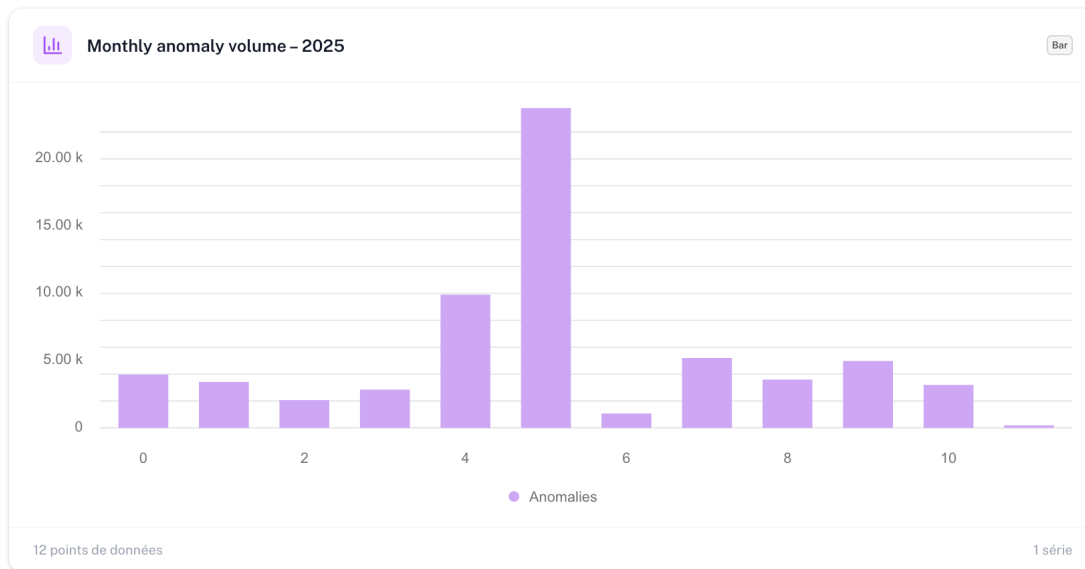


Figure 12: Automated generation of interactive bar charts

#### Anomaly landscape – 2025 (by typology)

| Month   | Total anomalies | Critical | Non-critical |
|---------|-----------------|----------|--------------|
| 2025-01 | 3 966           | 0        | 3 966        |
| 2025-02 | 3 421           | 0        | 3 421        |
| 2025-03 | 2 057           | 0        | 2 057        |
| 2025-04 | 2 841           | 0        | 2 841        |
| 2025-05 | 9 908           | 0        | 9 908        |
| 2025-06 | 23 783          | 0        | 23 783       |
| 2025-07 | 1 066           | 0        | 1 066        |
| 2025-08 | 5 190           | 0        | 5 190        |
| 2025-09 | 3 591           | 0        | 3 591        |
| 2025-10 | 4 971           | 0        | 4 971        |
| 2025-11 | 3 200           | 0        | 3 200        |
| 2025-12 | 1               | 0        | 1            |

*Trend:* the volume peaks in **June 2025** (23 783 anomalies) and is lowest in **July 2025** (1 066). The pattern is clearly seasonal, with a strong mid-year surge.

Figure 13: Tabular representation of the extracted anomaly data

Finally, the raw data is always coupled with an LLM-generated textual interpretation. This ensures that the user is not left alone with raw numbers, but is provided with actionable business insights and trend summaries.

#### Key performance indicators (KPIs)

- **Total anomalies (2025): 63 994**
- **Critical anomalies: 0**
- **Non-critical anomalies: 63 994**
- **Peak month (June): 23 783 anomalies**
- **Top typology count: 21 773** (Counterparty Ref Tethys)
- **Top 10 typologies share: 71 %** of all anomalies

#### Bar chart – Monthly anomaly volume (2025)

The chart shows the number of anomalies recorded each month, highlighting the strong June peak and the low July count.

*Interpretation:* The volume is highly seasonal; allocating extra monitoring resources around the mid-year period will address the bulk of the workload.

Figure 14: Textual interpretation and insight summary generated by the assistant

## 7 System Evaluation and Limitations

While the integration of an LLM-driven orchestration layer provides significant flexibility, deploying a Text-to-SQL system in a financial context requires a precise understanding of its operational boundaries. Our empirical testing highlighted both robust performance on standard business questions and clear limitations on more complex analytical requests.

### 7.1 Evaluation of Text-to-SQL Accuracy

The system was qualitatively evaluated using a predefined set of business questions provided by Natixis data stewards. For standard aggregations, such as anomaly counts by priority and monthly trend analyses, the constrained prompt consistently produced valid MySQL queries and correctly leveraged the expected tables, including `generic_anomalies` and `typologies`. In these scenarios, the assistant returned coherent indicators and concise business-oriented summaries.

The evaluation also showed that SQL generation quality is strongly dependent on the selected language model. Larger-parameter models were generally more reliable for multi-condition filters, JSON field extraction, and cross-table reasoning. By contrast, smaller local models sometimes struggled with complex temporal conditions or multi-step analytical logic. This observation confirms the importance of model selection in the interface and justifies the architecture choice that allows users to adapt model capacity to query complexity.

### 7.2 Architectural Limitations

Despite strong prompt constraints and a tool-based execution protocol, the current system remains primarily optimized for operational reporting use cases such as KPI extraction, category comparisons, and trend visualization. Very advanced analytical questions involving deeply nested subqueries, extensive historical windows, or complex SQL constructs can still exceed zero-shot generation reliability and may require human validation.

A second limitation concerns context scalability. Injecting broad schema information into the system prompt improves grounding, but it also consumes a substantial part of the model context window. As the database grows in size and semantic richness, a dynamic metadata retrieval strategy will become necessary to preserve accuracy while controlling latency and token usage.

A third limitation concerns production-grade security hardening. Although the prompt explicitly restricts behavior to read-only analysis and forbids destructive SQL operations, instruction-level safeguards alone are not sufficient for high-assurance environments. Future iterations should therefore enforce strict read-only database roles, add deterministic SQL validation gates before execution, and strengthen auditability across the full model-to-database interaction pipeline.

## 8 Conclusion

This project demonstrates that natural-language access to anomaly data can significantly improve operational efficiency in data-quality workflows. By combining a reproducible SQL data pipeline, a tool-based backend orchestration using Ollama and the AI SDK, and a user-centered interface, we built an assistant that remains both practical and controllable. Beyond faster access to indicators, the system helps align technical querying capabilities with business needs, which is essential for sustainable data governance in a regulated financial environment.